CS482 Software Project:
5-Card Draw Poker Game

Josue, Rohama, Preston, Ayana

2026-11-04

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Megan Olsen
- Client title: Owner
- Client email address: mmolsen@loyola.edu
- Client employer: N/A (Owner)
- How you know the client: Introduced by professor

2 Project Description

2.1 Overview

Our project is to create a 5 card draw poker game. Each player has fake currency that they can bet on in Poker to which they play against 2-6 players either with CPU or other players.

The client wants this project because the company earns revenue through ads, and they want us to create this game to promote external and internal ads.

2.2 Key Features

- User Login
- User Profile management
- Fake Currency system
- Ad system
- 5-card draw poker gameplay
- Home page, lobby, and music settings.
- Guest player Access
- Admin Access
- User database
- Admin interface

2.3 Why this Project is Interesting

This project was interesting for us because it can help us be a full stack developer where we create a software from scratch and we can apply what we learned from this past few years in to action. People should care about this project if they are looking at expanding their knowledge on how to play 5 card draw poker game because it's user friendly.

2.4 Areas of CS required

[What subfields of computer science seem most likely to be relevant to your project? A capstone must involve multiple.]

2.5 Potential Concerns and Questions

[Is there any aspect of this project that makes you unsure if it will work, either due to your own interests/background, or that you aren't sure if it fits the requirements? Are there questions you have about this project that you want instructor feedback about?]

2.6 Summary of Efforts to Find a Project

(Not necessary for 482) [Briefly list out when/how you've discussed with this client, and if you've discussed with other clients who either didn't work out or didn't respond. If you considered a different project and it didn't work out, why didn't it work out?]

[Most CS495 projects end here. The sections below are for CS482 and CS496 software projects].

2.7 Comparison to Draft

[For CS496 only, focus on highlighting the major differences between the draft proposal in CS495 and this one here. If there are no major differences, you can remove this subsection.]

3 Requirements

3.1 Non-Functional Requirements

[Non-functional requirements are just as important as functional requirements. Dont forget to specify them.]

Table 1 presents the NFRs for this software project.

ID	NFR Title	Category	Description
NFR1	NFR Example 1	Usability	Description of the NFR (it does not follow a user story template)
NFR2	NFR Example 2	Security	Description of the NFR (it does not follow a user story template)

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

[In CS482, all functional requirements are written as User Stories. In CS496, some projects may use a different template to write the requirements. The table below is an example of writing the Stories. Adapt accordingly to different templates or if you want to display more info.]

Table 2 presents the functional requirements for this software project.

ID	Story Title	Points	Description
S1	Story Example 1	5	As a user, I want to write a user story example, so that people will understand them.
S2	Story Example 2	2	As a user, I want to write a user story example, so that people will understand them.
	Total:	80	

Table 2: Functional requirements as User Stories.

3.3 AI Analysis on Functional Requirements

[After you receive your revised User Stories in CS482, or after your first delivery of Stories on CS496, you should use AI as a senior software engineer to help you find any missing or hidden requirements.] [You must provide which LLM you use, which prompt you used, and most importantly *reflect* on the results the LLM gave back. Just because the LLM suggested something, it does not mean it was a valid suggestion and that you should follow blindly. Use your knowledge and reason on which ones are reasonable to add to your requirements and which ones are not.]

4 System Design

4.1 Architecture

The type of software architecture our team will be following is MVC. This would be easier to incorporate in our project since we have a 5-card poker game. The MVC will let us keep the game logic, the screens, and the code that connects them separate. So for this game we will create several MVCs for different entities or responsibilities, M is model in which it will handle all the logic or calculations, V is view for the user to see their data, and C is for controller in which it is the middleman for M and V so it will pass the data along each other or be an API essentially. Examples of our modules would follow:

1. **Player management module** (Player, Admin): Handles sign up, login, guest access, and profile editing.
2. **Poker Gameplay module** (Player, Poker game, Table, Transaction): Runs a hand of 5-card draw, including dealing, betting, drawing, picking the winner, and updating chips.
3. **Lobby and Matchmaking module** (Table, Player, Music, Bots): Lets players browse, create, and join tables, and fills empty seats with bots.
4. **Complaint form module** (Account, Admin, chat logs, complaint form): Lets players report others and lets admins review reports with the chat logs.
5. **Admin Analytic module** (Admin, analytics): Shows admins active users, table counts, and ad revenue.
6. **Game Chat module** (Player, Chat): Sends live messages between players at a table and filters bad words.
7. **Settings module** (Account, Music): Saves each user's music, volume, and mute choices.
8. **Advertisement module** (Admin, Ads, Analytics): Shows ads, gives chip rewards for watching them, and tracks clicks.

4.2 Diagrams

4.3 Technology

We chose the MERN stack (MongoDB, Express, React, and Node.js) because it uses JavaScript which we are more familiar with and we will be able to work on the front and back end without switching between multiple languages. We also have used a lot of the tools required in previous classes which will make progress more smooth.

- **Main Programming Language: JavaScript/Typescript:** JavaScript is easy to help build components for our front end and easy to use for our server side/backend as well since MERN consists of just JavaScript, not having to switch between other languages makes it easier to use.
- **Testing: Jest:** As we used in class, Jest will be our testing framework since we are using Javascript, it is also because it is easy to create test cases with simple functions and able to create different tests via these functions.
- **Backend: Node.js + Express :** we decided to use Node.js and Express since it is very easy to run a Node.js server and for our routing, we also have experience using these 2 before.
- **Database: MongoDB, Mongoose:** In our database, we decided to go with MongoDB since it is easy to set up, it is a NoSQL database which we think would work well with our poker game and in JavaScript, we are already used to writing and using JSON. We will also use mongoose to format our data so that we can store the formatted data into our database.
- **Front end framework: React, Bootstrap:** For our front end, we decided to use React since it lets us build the UI out of reusable components. We will use Bootstrap, a separate CSS library, alongside React to help make our website look very nice, and Bootstrap Icons will give us icons to make our styles look pretty and unique.
- **Real-Time Communication: Socket.IO:** Since poker is played live with other players, everyone at a table needs to see bets, cards, and chat messages right away. Socket.IO keeps a connection open between the server and each player so updates show up instantly without refreshing the page.
- **Security: bcrypt:** We will use bcrypt to hash passwords before saving them, so real passwords are never stored in our database.

4.4 Coding Standards

Our team will follow standards from Extreme Programming (XP). We picked XP because it focuses on simple code, frequent testing, and teamwork, which fits a small team working in short sprints. We also added our own rules for the database. Our standards are grouped below.

1. For our **Designing Process** we will follow:
    - **Simplicity:** Making our code simple without extra content.
    - **System Metaphors**: creating scenarios that are equivalent to what we want to make to break down the problem
    - **CRC design**: Create class for our entities, their responsibilities so attributes, possibly methods, and collaboration so inheriting parent classes.
    - **Building linearly:** we linearly build so making sure each part of a system works in order to have a better and simpler system.
2. For our **Coding Standards**, we will use the following:
    - **Valid name conventions:** Our variables should make sense and are concise for the developers to understand.

- **Data privacy:** Information is discreet where needed to be, such as attributes in classes.
- **Code Clarity:** Frequently checking each other's codes and explaining to our teammates what it does and how it was done.
- **Team Reliability:** Solo programming is okay, but the team being available to help when needed when crossing an issue.
- **Test Driven programming:** Code should first be tested in order for it to be able to be pushed into the main branch.
- **Code responsibility:** if you broke the whole program, it is your responsibility to fix it.

3. For our **Testing Standards**, we will use the following.

   - **Testing Type:** What we will use for the majority is unit testing and mocking for our integration.
   - **Passing Tests and Coverage:** Our test cases should all pass before code is pushed into the main branch, and should have about 70-80% coverage of the code.
   - **Testing Simplicity:** Writing small and rapid test that test basic logic so that we can build on top of it for our bigger user stories.
   - **Mocking:** More into our mocking, we should use mocking when we are doing integration testing so we make sure that our data flow follows through with dummy data.

4. For our **Database sections**, we will use the following:

   - **Primary Keys:** Every document will use the unique ID that MongoDB creates automatically, which is a random mix of numbers and letters. Key fields will be named with "ID" at the end, like userID or tableID.
   - **Foreign Keys:** When one collection needs data from another, it will store that document's ID instead of copying the data.
   - **Organization:** We should use mongoose to organize how our data will be stored for each collection. Also should be used to store specific fields into the corresponding collection.

4.5 Data

For our data, since we are using MongoDB, our data is stored in collections instead of tables. Collections connect by storing the ID of another collection's document. The collections and their fields are listed here:

1. **Account:**

   - **PK — User ID:** The ID of the Account.
   - **Name:** The Person's Name.
   - **Email:** The Person's email.
   - **Username:** The username of the account.
   - **Display Name:** The nickname for the Account.
   - **Password:** The hashed password of the account, so the real password is never stored.
   - **Chip Amount:** The total chip amount an account has.
   - **Icon:** The account's figure to be shown to other players.
   - **Wins:** The account's number of wins.
   - **Losses:** The account's number of losses.
   - **Role:** Whether the account is a Player or an Admin.
   - **Settings:** The user's music choice, volume, and mute option.

2. **Music:** Where music options are stored.

- **PK — Music ID:** The ID for the music.
- **Music Type:** The song we want to play.
- **Song Name:** The name of the song.

3. **Bot:** To play against other players.
   - **PK — Bot ID:** The ID of the bot that played.
   - **Bot Name:** The name of the bot.
   - **Chip Amount:** The chip amount.
   - **Icon:** The robot icon.
4. **Admin:** Inherits Account, stored in the Account collection with Role set to Admin.
   - **Role:** Determines the role of the user, in this case, Admin.
5. **Player:** Inherits Account, stored in the Account collection with Role set to Player.
   - **Role:** Determines the role of the user, in this case, Player.
6. **Guest:**
   - **PK — Guest ID:** The ID of the guest.
   - **Role:** Determines the role of the user, in this case, Guest.
   - **Chip Amount:** The amount of chips the Guest has.
7. **Table:** Where people are able to play their games.
   - **PK — Table ID:** The ID of the table.
   - **FK — Account IDs:** A list of the accounts playing at the table.
   - **FK — Poker Game ID:** Keeps track of each Game that has been played.
   - **FK — Bot IDs:** A list of the bots at the table, if applicable.
   - **Table Name:** The name of the table the user created.
   - **Hands Played:** How many hands have been played throughout the table being open.
   - **Max Players:** The most amount of players that can play in one table.
   - **Min Players:** The minimum amount of players that can play in one table.
   - **Access:** Whether the table will be public or private.
   - **Invite Code:** If private, there will be a code used to be able to join a table.
   - **Default:** Default settings for whatever the user wants for the table.
   - **Status:** If the game is in play or not.
   - **Seats Available:** The number of seats left in a table.
   - **Games Played:** The number of rounds that have been played for the game.
   - **Chip Amounts:** Amount of chips during gameplay.
   - **Stakes:** The type of stake the game will play.
   - **Blind Amount:** The blind that must be played.
   - **Minimum Bet:** The minimum that must be bet.
8. **Poker Game:**
   - **PK — Poker Game ID:** The PK of the game.
   - **FK — Table ID:** To assign which table will play this game.

- **FK — Account ID:** Which player will play this game.
- **Player:** Includes Account ID FK, chip bet: chip amount bet, type of play :fold, check and raise, deck: the current cards they have, odds: The odds they have to win.
- **Rounds:** NumRound, the round number it is, and State. This includes what the user did, the amount they placed, and what their play was.
- **Pot:** The total amount of chips at the end of the game.
- **Cards:** The deck of cards the player will use.

9. **Chip Transactions:**
   - **PK — Transaction ID:** The ID for a specific transaction.
   - **FK — Account ID:** To see which account had the transaction.
   - **Amount:** The amount of chips possibly removed or added.
   - **Reason:** The reason to which their amount was altered.
   - **Date:** The date that this was made.
10. **Ads:**
    - **PK — Ad ID:** The ID of the ad.
    - **Title:** The title of the ad.
    - **Location:** Where the ad was.
    - **Chip Reward:** The amount of chips for watching this ad.
    - **Ad Type:** Internal or external ads.
    - **Ad Clicked:** The amount of times the ad was clicked.
11. **Chat Logs:**
    - **PK — Chat ID:** The ID for the chat.
    - **FK — Game ID:** In which game the chat had.
    - **FK — Account ID:** Who said the message.
    - **Message:** What was said in the game.
    - **Date:** The date on which it was said.
    - **Filter:** Censors the bad words.
12. **Admin Analytics:**
    - **PK — Analytics ID:** The ID of the analytics.
    - **FK — Account ID:** All the accounts for the game.
    - **FK — Ads ID:** All the ads we have and that have been clicked x amount of times.
    - **date** The stamp of the day.
    - **Users Active:** The amount of users currently active.
    - **Revenue:** How much money is generated by the ads.
    - **Table Creations:** How many tables were created.
13. **Complaint Form:**
    - **PK — Complaint ID:** ID for the complaint form.
    - **FK — Account ID:** The account that made the complaint.
    - **FK — Reported Player ID:** The player that was reported.

- **Date:** The time the form was submitted.
- **Content:** What was said about the report.

14. **Game history:**
    - **PK — Game History ID:** ID for the Game History.
    - **FK — Account ID:** The accounts that played during the game
    - **FK — poker Game ID:** The player's that played and their hands.
    - **Date:** The time they played.
15. **Friend List:**
    - **PK — Friend log ID:** ID for the Friend List.
    - **FK — Account ID:** The account that owns this friend list.
    - **FK — Friend IDs:** A list of account IDs that are friends with this account.

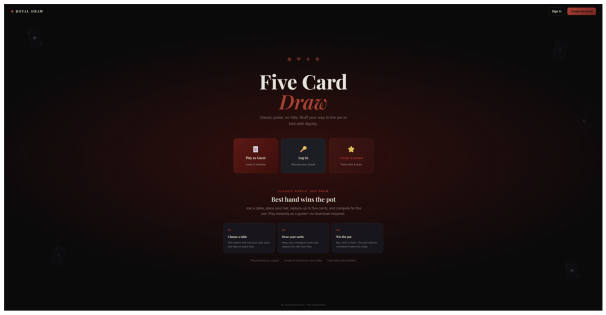
Here is an example of what one Account document would look like in the database:

```
{
  "_id": "665f1a2b9c8e4d0012ab34cd",
  "name": "Jane Doe",
  "email": "jdoe@example.com",
  "username": "jdoe",
  "displayName": "AceHigh",
  "password": "$2b$10$Xy...hashed...",
  "role": "player",
  "chipAmount": 2500,
  "icon": "icons/spade.png",
  "wins": 12,
  "losses": 9,
  "settings": { "music": "Lofi Beats", "volume": 60, "muted": false }
}
```

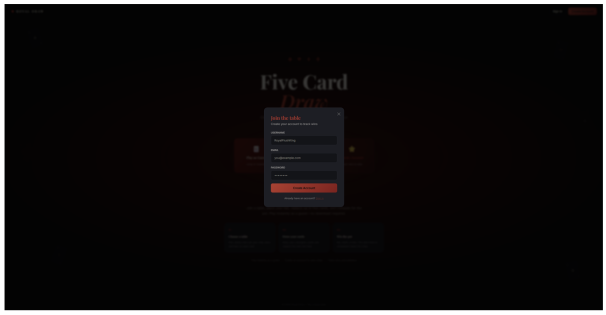
4.6 UI Mocks

Our UI mockups were made in Figma. Figure 1 shows the main screens a user will see. The home page lets users play as a guest, log in, or create an account. The create account and sign in forms collect the user's username, email, and password. The game lobby shows all open tables, lets players filter tables by stakes, and has an ad panel where players can watch ads to earn free chips.

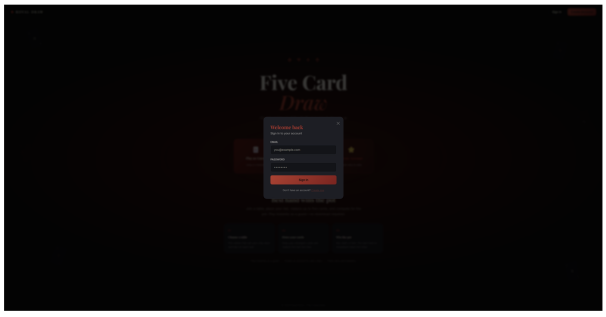
1. Home page



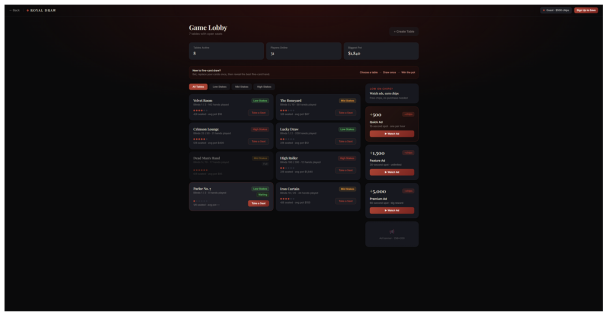
2. Create account form



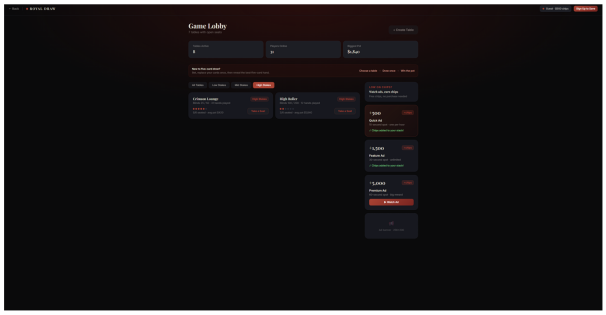
3. Sign in form



4. Game lobby (all tables)



5. Lobby filtered by high stakes, ad reward claimed



6. Lobby filtered by low stakes

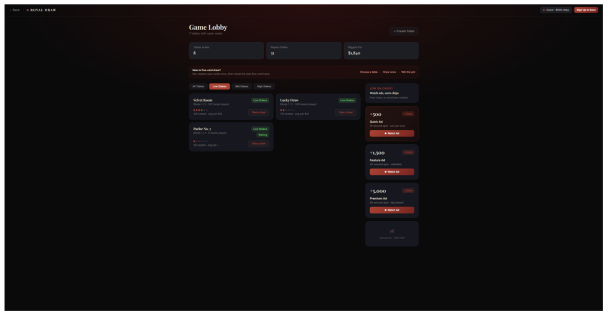


Figure 1: UI mockups for the main screens of our poker game.

5 Iterations

5.1 Iteration Planning

[In CS496, you plan all iterations beforehand. In CS482, you update the planning here at each iteration.]

Table 3 shows the iteration planning.

			Points	
It.	**Dates**	**Stories**	**Planned**	**Done**
1	10/07 - 10/22	G1 Guest without profile, V2 Visitor Account Creation, M2 Member password reset, A2 Admin Password Reset, P3 Player Spectating, P4 Player Mute Music, P7 Player Join Game, P9 Player Complaint form, P14 Player Table Creation	18	12
2	02/01 - 03/01	S3 Story Title, S4 Story Title, S5 Story Title, S6 Story Title	17	15
3	03/01 - 04/01	S7 Story Title, S8 Story Title, S9 Story Title, S10 Story Title, S11 Story Title	21	20
4	04/01 - 05/01	S12 Story Title, S13 Story Title, S14 Story Title, S15 Story Title	19	19
5	05/01 - 06/01	S16 Story Title, S17 Story Title	06	06
		Total:	70	65

Table 3: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

- G1: Guest without profile[1 point] - Preston
- V2: Visitor Account Creation[2 points] - Josue
- M2: Member password reset[1 points] - Ayana
- A2: Admin Password Reset[1 point] - Ayana
- P3: Player Spectating[2 points] - Preston
- P4: Player Mute Music[1 points] - Ayana
- P7: Player Join Game[1 points] - Preston
- P9: Player Complaint form[1 points] - Ayana
- P14: Player Table Creation[8 points] - Josue and Rohama

5.2.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

- Josue: The work I completed in the iteration was half the Table CRUD (P14), specifically the update and delete. The other story I worked on was the visitor sign up (V2) that needs a small implementation. The way I worked on the visitor sign up was implementing a SignUpView in which the user can input their name, email, username and password. We also have a Model and Controller for SignUp. A field that I am missing and will add to the next iteration is the confirm password field which is very important, but once someone signs up, they are logged in and redirected into a lobby in which the user can create tables which leads me to the Table CRUD. For the Table CRUD, Rohama and I worked on this in when she creates a table, I am able to update and delete the table. Once a table is created, only the host has access to their own table in which it shows an edit button to which you edit your table. Also with this, you are able to delete a table which is under the edit button which is straight forwards. For the table we have a TablceCreationView ,Model, and Creation.

- Rohama: I built the Create and Read parts of the table feature. For Create, I made a table creation form where the host picks the table name, stakes, number of seats from 2 to 6, whether it's private, and whether to add CPU players and how many. That goes to a backend endpoint, which validates it, saves it to MongoDB, and returns the new table. For Read, the lobby loads all public tables from the database, lets you filter them by stakes, and shows each table's seats, blinds, and status. I followed the same Model, DAO, Controller structure as our account code, and I wrote unit tests for the controller, the table form, and the lobby.

- Ayana: During this iteration, I focused on four user stories: member password reset, admin password reset, player mute music, and player complaint form. Initially, I considered all of these stories fully completed because my backend code was written and all of my test coverages passed. However, after UI testing, I would classify the member password reset and admin password reset as partially completed. While the player complaint form is thoroughly completed and tested, and the player mute music story is finished based on our initial requirements (though it may need a future touch-up once we consult our client, Dr. Olsen, regarding her audio preferences for the poker game), the password resets hit integration roadblocks. I had to wait for the sign-in functionality to be finalized before I could test them on the live UI, where I encountered an "invalid token" error on member reset and a missing interface view for admin resets. The core implementation and test coverage are solid, but the frontend integration and UI views need final debugging to transition them from partially completed to thoroughly completed.

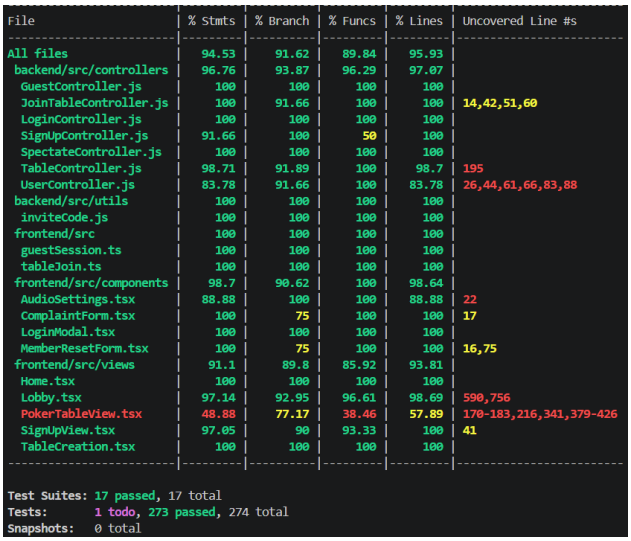
- Preston: The work I completed in this iteration was Guest w/o Profile (G1), Player Join Game (P7), and Player Spectating (P3). After our debrief I also took on Player Login. For G1, a guest can click Play as Guest and gets a guest account with 500 chips without signing up, and it gets deleted after 24 hours so their data isn't saved. For P7, tables now keep track of who is sitting in each seat, private tables get an invite code that only the host can see, and players can join a table as a member or a guest. For P3, players can watch any table, even a full one, and take a seat if one opens up. For Login, I connected the Log In button to Better Auth so members can sign back in with their email and password. I followed the same Model, DAO, Controller structure as the rest of our code, with a GuestModel, GuestController, JoinTableController, SpectateController, and LoginController. I used Claude to help write a lot of the code and tests, then tested everything on our database and merged it with the team's work.

5.2.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

- Our testing is good enough because strong test coverage and software quality, with 273 tests passing across all 17 test suites. The project achieved 94.53 statement coverage, 91.62 branch coverage, 89.84 function coverage, and 95.93 line coverage, indicating that most of the application's code and decision paths are exercised by automated tests. Several backend controllers, utilities, and frontend components achieved 100 coverage across all metrics. However, PokerTableView.tsx has lower coverage at 57.89 for lines, making it a priority for future testing improvements. Overall, these results provide a strong foundation for application reliability, while additional tests for uncovered code and the remaining todo test could further improve quality.

Figure 2 shows the test coverage



File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	94.53	91.62	89.84	95.93	
backend/src/controllers	96.76	93.87	96.29	97.07	
GuestController.js	100	100	100	100	
JoinTableController.js	100	91.66	100	100	14,42,51,60
LoginController.js	100	100	100	100	
SignUpController.js	91.66	100	50	100	
SpectateController.js	100	100	100	100	
TableController.js	98.71	91.89	100	98.7	195
UserController.js	83.78	91.66	100	83.78	26,44,61,66,83,88
backend/src/utils	100	100	100	100	
inviteCode.js	100	100	100	100	
frontend/src	100	100	100	100	
guestSession.ts	100	100	100	100	
tableJoin.ts	100	100	100	100	
frontend/src/components	98.7	90.62	100	98.64	
AudioSettings.tsx	88.88	100	100	88.88	22
ComplaintForm.tsx	100	75	100	100	17
LoginModal.tsx	100	100	100	100	
MemberResetForm.tsx	100	75	100	100	16,75
frontend/src/views	91.1	89.8	85.92	93.81	
Home.tsx	100	100	100	100	
Lobby.tsx	97.14	92.95	96.61	98.69	590,756
PokerTableView.tsx	48.88	77.17	38.46	57.89	170-183,216,341,379-426
SignUpView.tsx	97.05	90	93.33	100	41
TableCreation.tsx	100	100	100	100	

Test Suites: 17 passed, 17 total
Tests: 1 todo, 273 passed, 274 total
Snapshots: 0 total

Figure 2: Iteration 1 test coverage report

5.2.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

- Josue: After finishing up this iteration I had a lot of issues trying to understand MVC at first and organize our React code. We used Figma to create a mockup website, it had created a giant blob of code in which I had to split up into our view sections and have our App only call specific views when needed. I also had an issue signing up and staying logged in, this made me realize that I need a framework to create a session token for each player in which I used better-auth. A challenge I faced than was connecting the Better Auth into my current account model for a specific user but I was able to tie this in my referencing the ID for it. I think the sign up MVC did go according to plan except that I forgot to implement a confirm password field and for the Table CRUD it did as well, it was pretty straight forward. How I can avoid these mistakes is to a planning guide before hand and add logically what I need in a chronological order.

- Rohama: There wasn't much issue in my part as much but one of the problems I faced was the figma UI interface was a lot of code to past in to our own and it was a bunch to deal with. Other than that it was smooth on my part.

- Ayana: During this iteration, my main pitfall centered around the member and admin password resets. While test coverage looked solid, I noticed critical issues prior to the live demo. First, the admin

password reset option was not displayed on the UI because we haven't built out a dedicated admin page or section yet. Second, when testing the member password reset, clicking "forgot password" threw an "invalid token" error which suggests an issue with how tokens are generated, validated, or tied to our MongoDB schema. To fix this moving forward, I plan to build out the dedicated admin page, implement the UI so the admin password reset option renders correctly, and debug for both member and admin password resets. This iteration taught me a valuable lesson about dependency management. I need to ensure user stories are truly independent before picking them up, or coordinate much more closely with team members working on related functions. I can avoid this same mistake in future sprints by mapping out cross-story dependencies early during planning.

- Preston: The biggest challenge I had was getting my setup working, not the code itself. I had to install Node, Git, and MongoDB, and then found out my local MongoDB didn't work with Better Auth because sign up needs transactions, so I switched to our shared Atlas database. I also ran into merge conflicts when I pulled in Josue's changes to side, but they were small and I kept both of our changes. How I can avoid these issues next time is to check what the team is using before setting things up. Something I learned was how mocking works in Jest, which lets you test the controllers without needing a real database.

5.3 Iteration/Sprint 2

5.3.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.3.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.3.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.3.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.4 Iteration/Sprint 3

5.4.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.4.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.4.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.4.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.5 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482]

5.5.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.5.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.5.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.5.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project. Be concrete about your progress, you know how many story points your software is, how many points you completed (this shows your progress). You also how many points your team delivers at each iteration, therefore you can estimate how many more iterations it would take to finish the leftover points (show the math).]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently? How can you do better work in future projects? You may write this as a team or per person (or both — if all your iterations were team reflections, then it would be better to write individual reflections here)]

Appendix

[Appendix section if needed]